

TP4 -- Gestion memoire et introduction au C++

Matiere : Decouverte des langages compiles

Niveau : Bachelor 1

Duree : 4h (cours + atelier)

Jour : 4/5 — lundi 8 juin 2026

Objectifs pedagogiques couverts :

- Distinguer la pile (stack) et le tas (heap)
- Allouer dynamiquement avec malloc/calloc/realloc et liberer avec free
- Identifier les bugs memoire courants (fuite, segfault, double free)
- Compiler du C++ avec g++ et utiliser std::cout / std::cin
- Definir une classe avec attributs prives et methodes publiques
- Utiliser un constructeur et un destructeur pour gerer les ressources

Contexte

La gestion memoire est le sujet le plus delicat du C -- et c'est aussi ce qui en fait sa puissance. Vous allez comprendre ou vivent vos variables, comment en allouer plus a la demande, et pourquoi un programmeur C peut produire de magnifiques bugs.

Ensuite, on franchit le pont vers C++ : un langage qui ajoute la programmation orientee objet, une bibliotheque standard plus riche, et des constructions modernes -- tout en restant compatible avec le C.

Partie 1 -- Stack vs Heap : deux mondes (45 min)

1.1 Anatomie memoire d'un programme

+-----+ Stack (Pile)	<- Variables locales, par défaut
automatique, ~8 Mo, grandit ↓	Libérée automatiquement
+-----+ ...	
+-----+ Heap (Tas)	<- malloc / new
manuel, taille = RAM disponible	Doit être libérée avec free
+-----+ Données statiques	<- Variables globales, constantes
+-----+ Code (text)	<- Instructions du programme
+-----+	

1.2 Variables sur la stack

```
#include <stdio.h>

void afficher(void) {
    int a = 10;
    int b = 20;
    int c = 30;
    printf("a : valeur=%d, adresse=%p\n", a, (void*)&a);
    printf("b : valeur=%d, adresse=%p\n", b, (void*)&b);
    printf("c : valeur=%d, adresse=%p\n", c, (void*)&c);
    // a, b, c sont détruites automatiquement à la sortie de la fonction
}

int main(void) {
    afficher();
    afficher(); // les adresses peuvent être réutilisées !
    return 0;
}
```

Questions :

1. Les adresses de a, b, c sont-elles consecutives ? Quelle est la difference entre elles ?
2. Dans quel sens la stack grandit-elle sur votre machine ?
3. Que devient la variable `a` une fois la fonction `afficher` terminee ?

1.3 Allocation dynamique sur le heap : malloc et free

```
#include <stdio.h>
#include <stdlib.h>    // pour malloc et free

int main(void) {
    int n;
    printf("Combien de notes ? ");
    scanf("%d", &n);

    // Reserver n * sizeof(int) octets sur le tas
    int *notes = (int *) malloc(n * sizeof(int));
    if (notes == NULL) {
        printf("Echec d'allocation\n");
        return 1;
    }

    // Remplir et utiliser
    for (int i = 0; i < n; i++) {
        notes[i] = 10 + i;
    }

    // Afficher
    for (int i = 0; i < n; i++) {
        printf("notes[%d] = %d\n", i, notes[i]);
    }

    // RENDRE la memoire au systeme
    free(notes);
    notes = NULL;    // eviter de reutiliser un pointeur libere
    return 0;
}
```

Regles d'or :

- Tout `malloc` se termine par un `free`
- Toujours verifier si `malloc` retourne `NULL`
- Mettre le pointeur a `NULL` apres `free`

1.4 Variantes : calloc et realloc

Fonction	Role	Exemple
<code>malloc(t)</code>	Alloue t octets (contenu indefini)	<pre>int *p = malloc(10 * sizeof(int));</pre>
<code>calloc(n, t)</code>	Alloue n*t octets initialises a 0	<pre>int *p = calloc(10, sizeof(int));</pre>
<code>realloc(p, t)</code>	Redimensionne le bloc pointe par p	<pre>p = realloc(p, 20 * sizeof(int));</pre>

1.5 Exercice : tableau dynamique

Creez `tableau_dynamique.c` qui :

1. Demande a l'utilisateur combien d'entiers il veut stocker (n)
2. Alloue un tableau de n entiers sur le heap avec `malloc`
3. Demande a l'utilisateur de saisir chaque valeur
4. Affiche toutes les valeurs
5. Calcule et affiche la somme et la moyenne
6. Libere la memoire

Partie 2 -- Les bugs memoire classiques (30 min)

2.1 Les 5 ennemis

Bug	Description	Consequence
Fuite memoire	malloc sans free	Memoire perdue jusqu'a la fin du programme
Segfault	Acces a une adresse interdite (*p quand p == NULL)	Crash immediat
Double free	free(p) appele deux fois sur le meme pointeur	Comportement indefini
Use after free	Utiliser un pointeur apres l'avoir libere	Donnees aleatoires ou crash
Buffer overflow	Ecrire au-dela de la taille allouee	Corruption memoire

2.2 Exercice : trouver les bugs

Pour chaque programme, identifiez le bug et proposez une correction :

Programme A -- Buffer overflow :

```

#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int *tab = (int *) malloc(5 * sizeof(int));
    for (int i = 0; i <= 5; i++) {    // Indice suspect ?
        tab[i] = i * 10;
    }
    free(tab);
    return 0;
}

```

Programme B -- Dangling pointer :

```

#include <stdio.h>
#include <stdlib.h>

int* creer_tableau(int n) {
    int tableau[n];    // Variable locale !
    for (int i = 0; i < n; i++) tableau[i] = i;
    return tableau;    // On retourne l'adresse d'une variable
}

int main(void) {
    int *t = creer_tableau(5);
    printf("%d\n", t[0]);    // Que se passe-t-il ?
    return 0;
}

```

Programme C -- Buffer overflow sur chaine :

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
    char *nom = (char *) malloc(5 * sizeof(char));
    strcpy(nom, "Alexandre");    // "Alexandre" = 9 chars + '\0'
    printf("%s\n", nom);
    free(nom);
    return 0;
}

```

2.3 Detecter avec les sanitizers

Si Valgrind n'est pas disponible, compilez avec les sanitizers de GCC :

```
gcc -fsanitize=address -g programme.c -o programme
./programme
```

Exercice : Compilez le Programme A avec les sanitizers. Lisez et interprétez le message d'erreur.

Partie 3 -- Du C au C++ (45 min)

3.1 Hello World en C++

```
// hello.cpp
#include <iostream>
#include <string>

int main() {
    std::string nom;
    std::cout << "Quel est ton nom ? ";
    std::cin >> nom;

    std::cout << "Bonjour " << nom << " !" << std::endl;

    int age = 19;
    std::cout << "Tu as " << age << " ans" << std::endl;

    return 0;
}
```

Compilation C++ :

```
g++ -Wall -std=c++17 hello.cpp -o hello
```

3.2 Differences clés C → C++

Fonctionnalité	C	C++
Affichage	<code>printf("%s", s)</code>	<code>std::cout << s</code>
Saisie	<code>scanf("%d", &x)</code>	<code>std::cin >> x</code>
Chaines	<code>char[]</code> + <code>string.h</code>	<code>std::string</code>
Allocation	<code>malloc</code> / <code>free</code>	<code>new</code> / <code>delete</code>
Commentaires	<code>/* */</code>	<code>/* */</code> et <code>//</code>
References	Non	<code>int &ref = x;</code>

3.3 Les references (passage par reference)

```
#include <iostream>

void echanger(int &a, int &b) {    // & = passage par reference
    int tmp = a;
    a = b;
    b = tmp;
}

int main() {
    int x = 10, y = 20;
    std::cout << "Avant : x=" << x << ", y=" << y << std::endl;
    echanger(x, y);    // Pas besoin de & a l'appel, c'est tra
    std::cout << "Après : x=" << x << ", y=" << y << std::endl;
    return 0;
}
```

Comparez avec la version C du TP3 (pointeurs). La reference C++ simplifie la syntaxe.

3.4 Exercice : reecrir en C++

Recrivez le programme `fiche_interactive.c` du TP2 en C++ (`fiche.cpp`) :

- Utilisez `std::string` au lieu de `char[]`
- Utilisez `std::cout` / `std::cin` au lieu de `printf` / `scanf`
- Compilez avec `g++ -Wall -std=c++17`

Partie 4 -- Les classes en C++ (1h)

4.1 Concept : encapsuler donnees et comportements

```
// point.cpp
#include <iostream>

class Point {
private:           // attributs caches (encapsulation)
    double x;
    double y;
public:
    // Constructeur : appele a la creation d'un Point
    Point(double xi, double yi) { x = xi; y = yi; }

    // Methodes : actions sur l'objet
    double getX() const { return x; }
    double getY() const { return y; }
    void afficher() const {
        std::cout << "(" << x << ", " << y << ")" << std::endl;
    }
};

int main() {
    Point p1(3.0, 4.0);
    Point p2(10.0, 20.0);
    p1.afficher();    // (3, 4)
    p2.afficher();    // (10, 20)
    return 0;
}
```

Vocabulaire :

- **Classe** = modele (template) qui definit des attributs et des methodes
- **Objet** = instance d'une classe (une realisation concrete)
- **Constructeur** = fonction speciale appelee automatiquement a la creation
- **private** = accessible uniquement dans la classe
- **public** = accessible depuis l'exterieur

4.2 Constructeur et destructeur

```
#include <iostream>

class Compteur {
private:
    int valeur;
    int *historique;    // tableau dynamique
    int taille;
public:
    Compteur(int n) {                // CONSTRUCTEUR
        valeur = 0;
        taille = n;
        historique = new int[n];     // alloue sur le
        std::cout << "Compteur cree" << std::endl;
    }

    ~Compteur() {                    // DESTRUCTEUR
        delete[] historique;         // libere la memo
        std::cout << "Compteur detruit" << std::endl;
    }

    void incrementer() { valeur++; }
    int lire() const { return valeur; }
};

int main() {
    Compteur c(100);                // constructeur appele automatiqu
    c.incrementer();
    c.incrementer();
    std::cout << c.lire() << std::endl;
    return 0;
} // Destructeur appele AUTOMATIQUEMENT a la sortie
```

RAII (Resource Acquisition Is Initialization) : le constructeur acquiert les ressources, le destructeur les libere. C'est le grand atout du C++ sur le C.

4.3 Pourquoi encapsuler ?

- **Protéger les données** : si `age` est private, personne ne peut écrire `e.age = -50`
- **Cacher la complexité** : l'utilisateur de la classe voit les méthodes, pas l'implémentation
- **Faciliter la maintenance** : les bugs sont localisés dans la classe

4.4 Exercice : classe Etudiant

Créez `etudiant.cpp` avec une classe `Etudiant` :

```
class Etudiant {
private:
    std::string nom;
    int age;
    double moyenne;

public:
    // Constructeur avec liste d'initialisation
    Etudiant(std::string n, int a, double m);

    // Methodes
    void ajouterNote(double note);    // recalcule la moyenne
    bool estMajeur() const;          // return age >= 18
    void afficher() const;           // affiche nom, age, mo

    // Getters
    std::string getNom() const;
    double getMoyenne() const;
};
```

Dans `main()` :

1. Créer un tableau de 5 `Etudiant` (sur la stack ou avec `new`)
2. Saisir les infos de chaque étudiant au clavier

3. Afficher tous les étudiants
4. Calculer et afficher la moyenne de la promotion
5. Trouver et afficher l'étudiant ayant la meilleure moyenne

Si vous utilisez `new` , n'oubliez pas `delete[]` a la fin. Pensez RAII.

Partie 5 -- Bilan : tableau recapitulatif

Completez ce tableau :

Concept	Definition	Exemple en C/C++
Stack		
Heap		
<code>malloc</code>		
<code>free</code>		
Memory leak		
Dangling pointer		
Buffer overflow		
Classe		
Constructeur		
Encapsulation		

Pour aller plus loin

- Outil en ligne [Pythontutor.com](https://pythontutor.com) : visualise la memoire etape par etape
- `calloc` vs `malloc` : quelle difference ?
- Smart pointers en C++ : `std::unique_ptr` , `std::shared_ptr` (C++11)
-- alternative a `new` / `delete`
- Lecture recommandee : *The C Programming Language* (Kernighan & Ritchie), chapitres 5-6